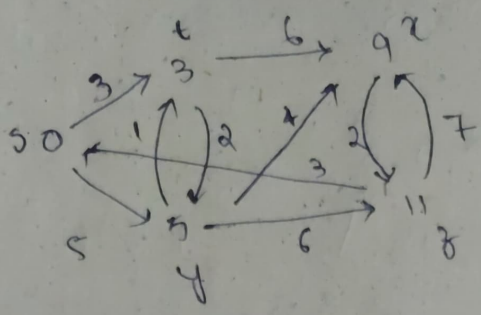
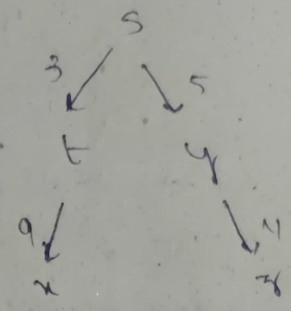
Tut-9

1)





adj-list

s → {t, 3}, {y, 5}

t → {x, 6}, {y, 2}

x → {z, 2}

z → {x, 7}, {s, 3}

y → {t, 1}, {x, 4}, {z, 6}

d =

0	~~3~~	~~9~~	~~5~~	~~11~~
s	t	x	y	z

Relaxation:

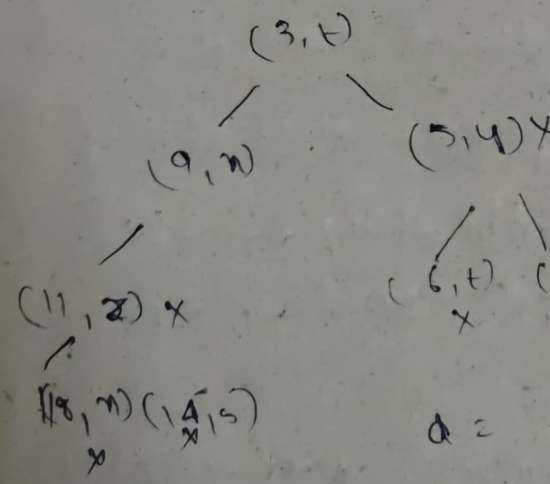
```
if ( d[u] + c(u,v) < d(v] )
        d[v] = d[u] + c(u,v)
```

2) using PQ

d =

s	t	y	x	z
0	~~3~~	~~5~~	~~9~~	~~11~~

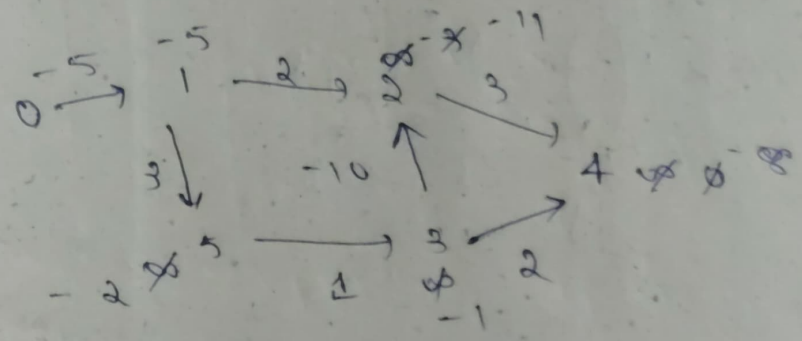
(11, z)
(9, x)
(5, y)
(3, t)
(0, s)



d = ⟨0, 3, 5, 9, 11⟩

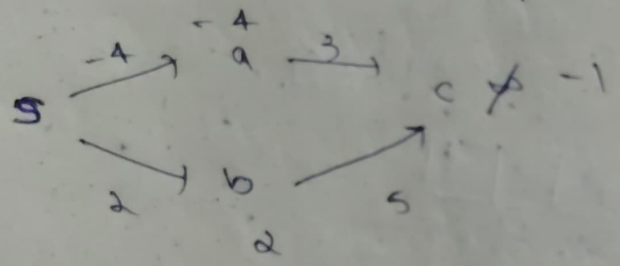
⟨s, t, y, x, z⟩

3) for negative weight edges Directed graph Dijkstra algorithm doesn't work



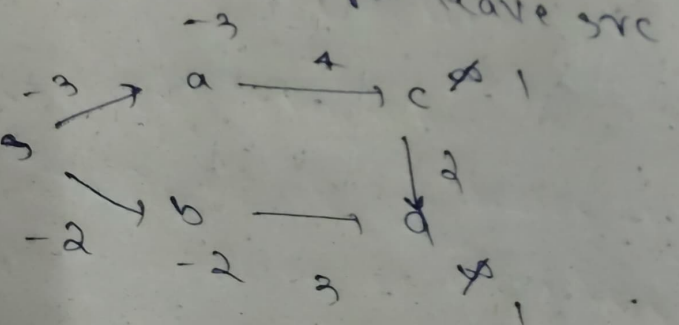
-11 is discovered after it is finalized to -3

4) a) exactly one -ve edge leaving src



* Distance can only decrease after the -ve edge never decrease later

b) All -ve edges leave src



* All -ve edges are handled at the start after that, only +ve edges remain no shorter path appears later.

* Dijkstra Algorithm fails only when a better path appears after a node is finalized if

all -ve edges leaving src (or) a single edge as -ve weight this doesn't fail at any time.

5) G → DAG {w ∈ IR}

a) shortest-path (G, s)

```
topo = topoSort(G)
for each vertex v
    d[v] = ∞
d[s] = 0
for u in topo:
    for each edge (u,v) with w
        if d[v] > d[u] + w
            d[v] = d[u] + w
return d
```

b) longest-path (G, s)

```
topo = topological sort(G)
for each vertex v
    d[v] = -∞
d[s] = 0
for u in topo
    for each edge(u,v) with w
        if d[v] < d[u] + w
            d[v] = d[u] + w
return d.
```

6) a) Floyd-warshall Algorithm

1. Do cost matrix

adj matrix → same edge = 0
no edge = ∞

2. Go via every node & update the matrix.
3. Apply 3 for loops of via, i, j
by checking condition path = min(
d[i][via] + d[via][j])
4. we get cost[i][j] for every edge.
5. check if cost[i][i] < 0 : -ve cycle exist.

Proof: cost[i][j] → shortest path from i to j
cost[i][i] → shortest path from i back to itself
General if there is no -ve cycle
then cost[i][i] = 0

b) Bellman-ford Algorithm:
In a graph of V nodes relaxation of all edges is done V-1 times sequentially. On the nth iteration, no relaxation will be done and if the dist[] array gets reduces then detects -ve cycles

BellmanFord(G, s):

for each vertex v

d[v] = ∞

d[s] = 0

for i = 1 to V-1

for each edge (u,v) with w

if d[v] > d[u] + w

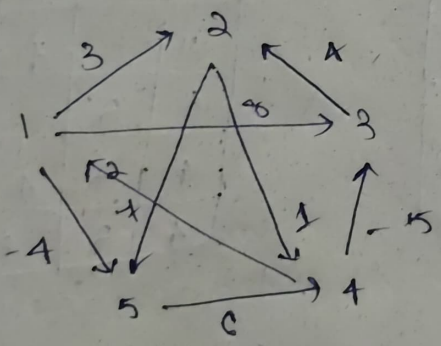
d[v] = d[u] + w

for each edge (u,v) with w

if d[v] > d[u] + w

return "negative cycle exists"

Ex



k	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

via 1 →

0	3	8	∞	-4
∞	0	∞	1	7
∞	4	0	∞	∞
2	5	-5	0	-2
∞	∞	∞	6	0

via 2 →

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	5	-5	0	-2
∞	∞	∞	6	0

via 3 →

0	3	8	4	-4
∞	0	∞	1	7
∞	4	0	5	11
2	-1	-5	0	-2
∞	∞	∞	6	0

via 4

0	3	-1	4	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

via 5

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

* predecessor matrix Π^k : how you reached a vertex in the shortest path

$\Pi[i][j]$: the vertex just before ~~path~~ i on the shortest path from i to j

$\Pi^{(0)}$ =

	1	1		1
			2	2
	3			
4		4		
			5	

$\Pi^{(5)}$

	3	4	5	1
4		4	2	1
4	3		2	1
4	3	4		1
4	3	4	5	

Path 1 → 3

$\Pi[1][3] = 4$

$\Pi[1][4] = 5$

$\Pi[1][5] = 1$

1 → 5 → 4 → 3